DOCUMENTATION · VALIDATION 1

SkyTrack
Architecture globale

Plateforme de surveillance et d'analyse météorologique en microservices

01
Vue d'ensemble
Clients, frontend, API Gateway, 6 microservices, bases de données et services transverses

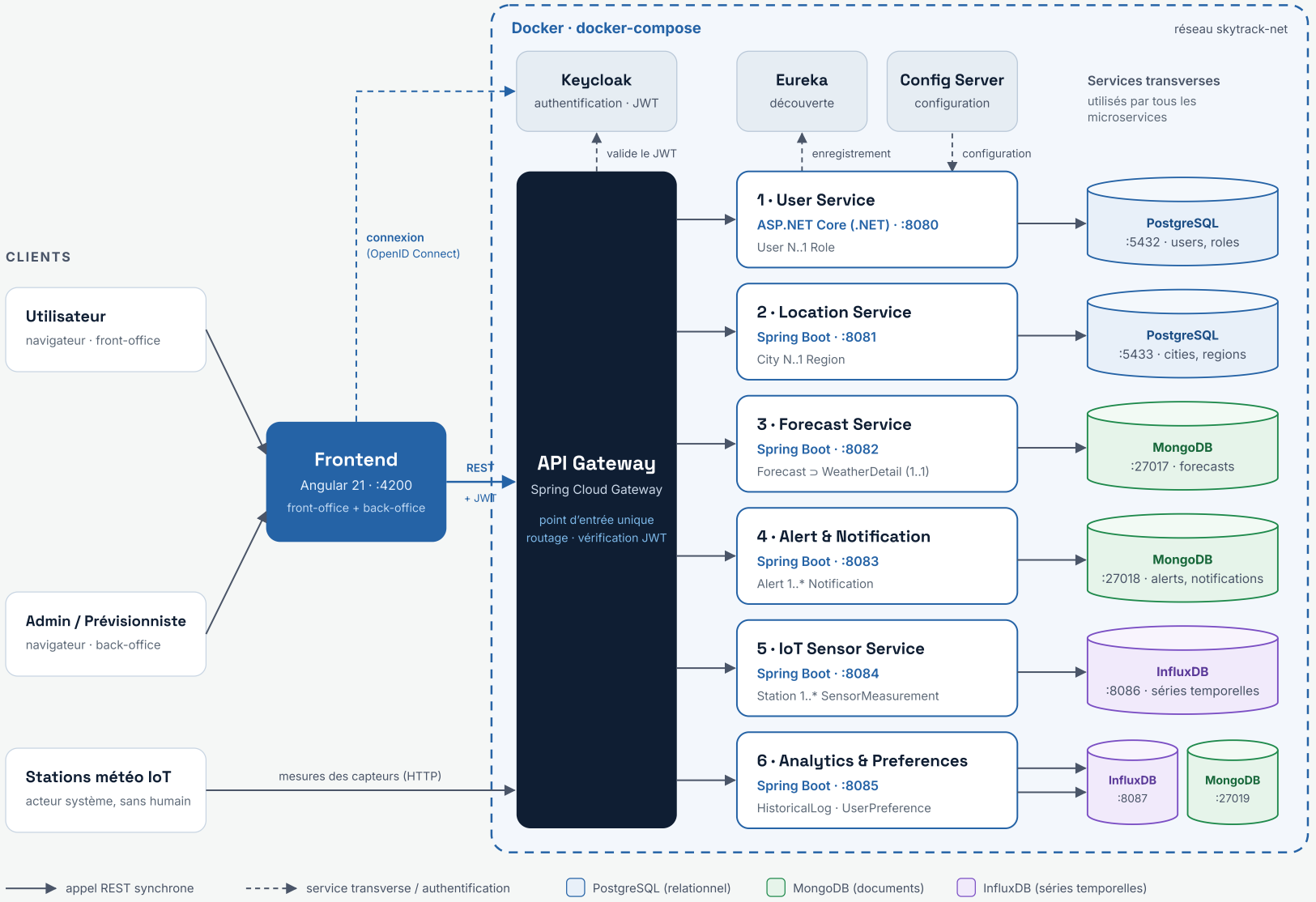
02
Communication inter-services
Appels synchrones OpenFeign, identifiants échangés, tolérance aux pannes

03
Déploiement Docker
Conteneurs, ports et réseau docker-compose

Équipe : Med Dhia Allagui · Yasmine Fehri · Ahmed Bouallegue · Abdallah Grissa · Yassine Wesslati · Salim Khaldi

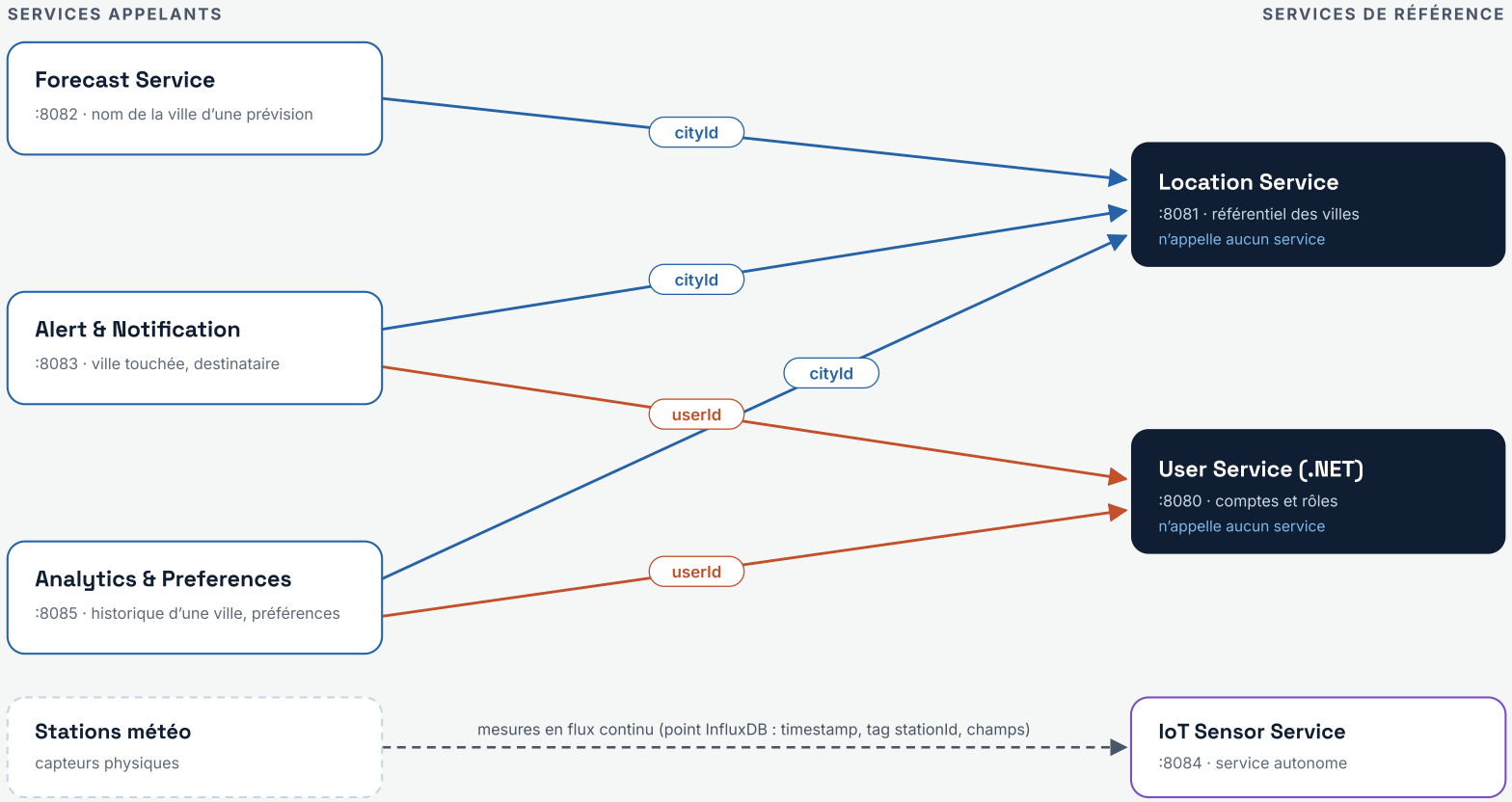
Architecture globale

Tout le trafic externe entre par l'API Gateway. Chaque microservice possède sa propre base de données ; aucune base n'est partagée. Les appels entre services sont détaillés page 02.



Communication inter-services

Les services n'échangent que des identifiants logiques. La flèche va du service qui appelle vers le service appelé, jamais l'inverse.



Synchrone, via OpenFeign

Chaque appel a besoin de la réponse immédiatement. Le service appelé est trouvé par son nom dans Eureka (ex. **location-service**), jamais par une adresse en dur. Le User Service en .NET est appelé en REST/JSON comme les autres.

Seulement des identifiants

cityId et **userId** sont des références logiques, pas des clés étrangères : aucun service ne lit la base d'un autre. Un *Long* en Java correspond à un *long* en C#.

Si un service tombe

Timeout + circuit breaker (Resilience4j) : réponse en mode dégradé au lieu d'une erreur en cascade. Exemple : une prévision reste lisible avec **cityName = null** si Location ne répond pas ; la création est refusée (503).

Déploiement Docker

Un conteneur par microservice et un par base de données, lancés ensemble par **docker-compose**. Dans les conteneurs, chaque base écoute sur son port par défaut ; seul le port côté machine hôte change.

user-service — Ahmed Bouallegue

- `8080:8080` ASP.NET Core (.NET)
- `5432:5432` postgres-user · PostgreSQL

location-service — Salim Khaldi

- `8081:8081` Spring Boot
- `5433:5432` postgres-location · PostgreSQL

forecast-service — Med Dhia Allagui

- `8082:8082` Spring Boot
- `27017:27017` mongo-forecast · MongoDB

alert-service — Yasmine Fehri

- `8083:8083` Spring Boot
- `27018:27017` mongo-alert · MongoDB

iot-service — Yassine Wesslati

- `8084:8084` Spring Boot
- `8086:8086` influx-iot · InfluxDB

analytics-service — Abdallah Grissa

- `8085:8085` Spring Boot
- `8087:8086` influx-analytics · InfluxDB
- `27019:27017` mongo-analytics · MongoDB

frontend

- `4200` Angular 21

api-gateway

- `à définir` Spring Cloud Gateway

eureka-server

- `8761*` Netflix Eureka

config-server

- `8888*` Spring Cloud Config

keycloak

- `8180:8080*` Keycloak

Réseau skytrack-net : tous les conteneurs sont sur le même réseau Docker et se joignent par leur nom (ex. `mongo-forecast:27017`). Seuls le frontend et l'API Gateway ont besoin d'être exposés à l'extérieur ; les ports des bases ne sont publiés que pour le développement.

Notation **hôte:conteneur**. Ports des microservices et des bases : README du projet. * Ports par défaut proposés pour l'infrastructure (Keycloak décalé en 8180 pour ne pas entrer en conflit avec le User Service) : à confirmer par l'équipe.